Top-K Frequent Words

Category: Heap Algorithms | Language: Python 3

1. Aim

Find the k most frequent words in a text file while keeping only k candidate words inside a min-heap. This demonstrates a practical use for a heap.

2. How the program works

The text is converted to lowercase and split into English alphabetic words. A dictionary stores the count of each word.

The program reads each unique (count, word) pair. If the heap has fewer than k items, it inserts the pair. Otherwise, it replaces the root only when the new pair is better. At the end, it sorts and prints the remaining k items. For equal counts, words later alphabetically win in this simple implementation.

3. Sample output (k = 5)

Rank	Word	Count
1	algorithms	5
2	data	3
3	and	3
4	useful	2
5	code	2

4. Complexity

Let t be the number of words, u the number of unique words, and k the requested number of results. Counting takes **O(t)** expected time using a dictionary. Heap operations for u distinct words cost **O(u log k)**, and sorting the final k items costs **O(k log k)**. Memory is **O(u + k)**.

5. Testing and conclusion

I tested basic word counts, the top two words, tied counts, empty text, and k = 0. All the unit tests passed.

Conclusion: A size-k heap is useful when k is much smaller than the number of distinct words because the program does not need to sort every word.